

Resoluções Didáticas e Científicas —

Capítulo 4 — Parte 1

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Estratégia de Controle: Divisão Estratégica (Parte 1: Exercícios de 4.1 a 4.15) para garantir 100% de abrangência, profundidade de Engenharia de Software e zero omissões devido a limites de tamanho.

Apresentação Pedagógica

Olá! É uma imensa satisfação dar as boas-vindas a você nas resoluções estruturadas do Capítulo 4. Neste capítulo, avançamos significativamente na programação estruturada ao introduzir os conceitos de **repetição controlada por contador simplificada** com a instrução **for**, a estrutura de seleção múltipla **switch**, o laço de repetição pós-teste **do...while** e as instruções de desvio de fluxo **break** e **continue**, além dos operadores lógicos compostos (**&&**, **||** e **!**).

Seguindo rigorosamente a nossa ****abordagem pedagógica incremental****, todas as soluções e códigos a seguir restringem-se estritamente ao conteúdo cumulativo até o Capítulo 4. Não faremos uso de funções customizadas, ponteiros ou arranjos (vetores), que pertencem aos capítulos seguintes. Além disso, conforme sua diretriz estabelecida, eliminamos a geração de blocos de sintaxe LaTeX brutos, focando exclusivamente na prosa explicativa e no código C limpo, completo e funcional.

1. Exercícios de Autorrevisão

Exercício 4.1 — Preenchimento de Lacunas (Estruturas de Controle)

- a) O laço controlado por contador simplificado é implementado em C através da instrução **for**.
- b) As instruções **break** e **continue** alteram o fluxo normal de execução de um laço de repetição.
- c) A estrutura de seleção múltipla que testa se uma variável corresponde a uma série de valores inteiros constantes é a instrução **switch**.
- d) O laço de repetição que garante a execução do seu corpo pelo menos uma vez antes de avaliar a condição de teste é a instrução **do...while**.
- e) Para combinar múltiplas condições em uma única expressão lógica, utilizamos os operadores lógicos compostos **&&** (**E lógico**) e **||** (**OU lógico**).

Exercício 4.2 — Verdadeiro ou Falso (Análise Teórica)

- a) **Falso**. A instrução **switch** pode testar apenas expressões que resultem em valores inteiros (como **int**, **char**, **short** ou enumerações). Expressões baseadas em números de ponto flutuante

(como float ou double) não são permitidas.

b) Falso. A instrução break encerra imediatamente a execução do laço ou bloco switch corrente, desviando o fluxo para a primeira linha de código após a estrutura. Ela não encerra o programa inteiro.

c) Verdadeiro. O operador lógico && possui um nível de precedência mais alto na tabela de operadores do que o operador lógico ||.

d) Falso. Se a condição de teste inicial de um laço while for avaliada como falsa, o corpo do laço nunca será executado.

e) Verdadeiro. O operador de negação lógica (!) é um operador unário e inverte o valor booleano da expressão associada.

Exercício 4.3 — Escrita de Instruções Isoladas

a) Somar os números ímpares entre 1 e 99 utilizando um laço for:

```
int soma = 0;
int i;
for (i = 1; i <= 99; i += 2) {
    soma += i;
}
```

b) Calcular e exibir o valor de 2.5 elevado à potência 3 usando a função pow:

```
/* Nota de Engenharia: A função pow exige a inclusão de <math.h> */
printf("Resultado: %.2f\n", pow(2.5, 3.0));
```

c) Imprimir os números de 1 a 20, inserindo uma nova linha a cada 5 números:

```
int x;
for (x = 1; x <= 20; x++) {
    printf("%d\t", x);
    if (x % 5 == 0) {
        printf("\n");
    }
}
```

2. Exercícios Regulares (Códigos de Engenharia de Software)

Exercício 4.9 — Somatório de uma Sequência de Inteiros

Enunciado: Escreva um programa que some uma sequência de inteiros. Suponha que o primeiro inteiro lido especifique o número de elementos restantes a serem inseridos pelo usuário.

```
#include <stdio.h>
```

```
int main(void) {
```

```

int total_elementos;
int valor, soma = 0;
int i;

printf("Digite a quantidade de números que deseja somar: ");
scanf("%d", &total_elementos);

for (i = 1; i <= total_elementos; i++) {
    printf("Digite o número %d: ", i);
    scanf("%d", &valor);
    soma += valor;
}

printf("\nA soma dos %d números informados é: %d\n",
total_elementos, soma);
return 0;
}

```

Exercício 4.10 — Cálculo da Média de uma Sequência Controlada por Sentinela

Enunciado: Escreva um programa que calcule e imprima a média de vários inteiros. Suponha que o último valor lido com `scanf` seja o sentinela 9999.

```
#include <stdio.h>
```

```

int main(void) {
    int valor;
    int soma = 0;
    int contador = 0;

    printf("Digite um inteiro (9999 para encerrar e calcular a média):
");
    scanf("%d", &valor);

    while (valor != 9999) {
        soma += valor;
        contador++;

        printf("Digite um inteiro (9999 para encerrar): ");
        scanf("%d", &valor);
    }

    /* Proteção contra divisão por zero */
    if (contador != 0) {
        printf("\nA média dos %d valores inseridos é: %.2f\n",
contador, (float) soma / contador);
    }
}

```

```

    } else {
        printf("\nNenhum valor válido foi inserido.\n");
    }

    return 0;
}

```

Dica de Prevenção de Erros 4.1: A operação de conversão explícita de tipo (cast) (float) soma converte temporariamente o valor inteiro em ponto flutuante antes da divisão, impedindo que o compilador realize uma divisão inteira truncada e preserve as casas decimais corretas da média.

Exercício 4.11 — Localização do Menor Inteiro em um Fluxo

Enunciado: Escreva um programa que localize o menor inteiro de um conjunto de valores. O primeiro valor lido especifica a quantidade de elementos que virão a seguir.

```
#include <stdio.h>
```

```

int main(void) {
    int quantidade, valor, menor;
    int i;

    printf("Informe a quantidade de valores a serem avaliados: ");
    scanf("%d", &quantidade);

    if (quantidade > 0) {
        printf("Digite o primeiro número: ");
        scanf("%d", &valor);
        menor = valor; /* Define o primeiro número como o menor
inicial */

        for (i = 2; i <= quantidade; i++) {
            printf("Digite o próximo número: ");
            scanf("%d", &valor);

            if (valor < menor) {
                menor = valor; /* Atualiza o menor valor encontrado */
            }
        }

        printf("\nO menor número inserido no conjunto é: %d\n",
menor);
    }

    return 0;
}

```

Exercício 4.12 — Somatório de Inteiros Pares em um Intervalo

Enunciado: Escreva um programa que calcule e imprima a soma dos inteiros pares de 2 a 30.

```
#include <stdio.h>
```

```
int main(void) {
    int soma = 0;
    int i;

    for (i = 2; i <= 30; i += 2) {
        soma += i;
    }

    printf("A soma de todos os inteiros pares entre 2 e 30 é: %d\n",
soma);
    return 0;
}
```

Exercício 4.13 — Produto de Números Ímpares

Enunciado: Escreva um programa que calcule e imprima o produto dos inteiros ímpares de 1 a 15.

```
#include <stdio.h>
```

```
int main(void) {
    int produto = 1;
    int i;

    for (i = 1; i <= 15; i += 2) {
        produto *= i;
    }

    printf("O produto de todos os inteiros ímpares entre 1 e 15 é:
%d\n", produto);
    return 0;
}
```

Exercício 4.14 — Cálculo Computacional de Fatoriais

Enunciado: Escreva um programa que calcule os fatoriais dos inteiros de 1 a 5. Exiba os resultados em formato tabular.

```
#include <stdio.h>
```

```
int main(void) {
    int fatorial;
```

```

    int i, j;

    printf("Número\tFatorial\n\n");

    for (i = 1; i <= 5; i++) {
        fatorial = 1;

        /* Laço interno para computar o fatorial do número i corrente
*/
        for (j = 1; j <= i; j++) {
            fatorial *= j;
        }

        printf("%d\t%d\n", i, fatorial);
    }

    return 0;
}

```

Exercício 4.15 — Sistema de Cálculo de Juros Compostos Modificado

Enunciado: Modifique o programa de juros compostos da Seção 4.6 para repetir os cálculos para taxas de juros de 5%, 6%, 7%, 8%, 9% e 10%. Exiba os resultados de forma organizada.

Fórmula Matemática: $A = P(1 + r)^n$

```
#include <stdio.h>
```

```
#include <math.h>
```

```

int main(void) {
    double principal = 1000.0; /* Capital inicial */
    double montante;
    int ano, taxa_percentual;
    double taxa_decimal;

    /* Laço externo que varia as taxas de juros de 5% a 10% */
    for (taxa_percentual = 5; taxa_percentual <= 10;
taxa_percentual++) {
        taxa_decimal = (double) taxa_percentual / 100.0;

        printf("\n--- Tabela de Rendimento para Taxa de %d%% ---\n",
taxa_percentual);
        printf("Ano\tMontante em Conta\n");

        for (ano = 1; ano <= 10; ano++) {
            montante = principal * pow(1.0 + taxa_decimal, ano);
            printf("%d\tR$ %.2f\n", ano, montante);
        }
    }
}

```

```
    return 0;  
}
```

As resoluções prosseguirão na Parte 2, iniciando a partir do Exercício 4.16.